

Clase 14

ABSTRACCIÓN FUNCIONAL (SEGUNDA PARTE)

CC1002 Introducción a la
Programación

Funciones anónimas

- Funciones “sin nombre”
- Pensadas como funciones que se utilizan una única vez
- Para definir funciones anónimas en Python se utiliza la instrucción **lambda**

```
lambda id1, id2, ...: expresion
```

CC1002 Introducción a la
Programación

Funciones anónimas

- Ejemplos:

```
lambda x, y: x + y
```

```
lambda x: x < 5
```

- Si la expresión se utiliza más de una vez, debe definirse como función usando la receta de diseño

CC1002 Introducción a la
Programación

Formalizando la abstracción funcional

- Requiere realizar los siguientes pasos:
 - Comparar las funciones
 - Abstraer las funciones
 - Implementar función abstracta
 - Validar que la abstracción es correcta

CC1002 Introducción a la
Programación

Comparación

- Comparar las funciones y marcar las diferencias:

```
# convertirCF: lista(num) -> lista(num)
# convierte los valores en unaLista de grados C a grados F
# ejemplo: convertirCF(lista(0, listaVacía)) devuelve lista(32, listaVacía)
def convertirCF(unalista):
    if vacía(unalista):
        return listaVacía
    else:
        return lista(CaF(cabeza(unalista)), convertirCF(cola(unalista)))

#ri: nombre (str) valor (int)

# nombres: lista(RI) -> lista(str)
# devuelve una lista de Strings con la lista de registros en unaLista
# ejemplo: nombres(lista(ri('ril', 1), listaVacía)) devuelve lista('ril 1', listaVacía)
def nombres(unalista):
    if vacía(unalista):
        return listaVacía
    else:
        return lista(RIaString(cabeza(unalista)), nombres(cola(unalista)))
```

CC1002 Introducción a la
Programación

Abstracción

- Reemplazar diferencias correspondientes con nuevos identificadores (agregar dichos nombres a los parámetros):

```
# convertirCF: lista(num) -> lista(num)
# convierte los valores en unaLista de grados C a grados F
# ejemplo: convertirCF(lista(0, listaVacía)) devuelve lista(32, listaVacía)
def convertirCF(F, unaLista):
    if vacía(unalista):
        return listaVacía
    else:
        return lista(F(cabeza(unalista)), convertirCF(cola(unalista)))

# nombres: lista(RI) -> lista(str)
# devuelve una lista de Strings con la lista de registros en unaLista
# ejemplo: nombres(lista(ri('ril', 1), listaVacía)) devuelve lista('ril 1', listaVacía)
def nombres(F, unaLista):
    if vacía(unalista):
        return listaVacía
    else:
        return lista(F(cabeza(unalista)), nombres(cola(unalista)))
```

CC1002 Introducción a la
Programación

Implementar función abstracta y validar

```
def mapa(f, unaLista):
    if vacia(unaLista):
        return listaVacia
    else:
        return lista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))

# convertirCF: lista(num) -> lista(num)
def convertirCF(unaLista):
    return mapa(CaF, unaLista)

# nombres: lista(RI) -> lista(str)
def nombres(unaLista):
    return mapa(RIaString, unaLista)

# Las nuevas versiones de convertirCF y nombres deben pasar todos
# los tests implementados para las versiones originales
```

CC1002 Introducción a la
Programación

Contrato

- Si vemos a `mapa` como una abstracción de **`convertirCF`**, el contrato sería:

```
# mapa: (num -> num) lista(num) -> lista(num)
```

- Por el contrario, si vemos a `mapa` como una abstracción de **`nombres`**, el contrato sería:

```
# mapa: (RI -> str) lista(RI) -> lista(str)
```

CC1002 Introducción a la
Programación

Contrato

- Para acomodar ambos casos, es necesario entender qué es lo que hace mapa y luego definir el contrato
- Contrato general de mapa:

```
# mapa: (X -> Y) lista(X) -> lista(Y)
```

CC1002 Introducción a la
Programación

Función mapa

- Finalmente, función mapa queda como:

```
# mapa: (X -> Y) lista(X) -> lista(Y)
# devuelve lista con funcion f aplicada a todos los valores de unaLista
def mapa(f, unaLista):
    if vacia(unaLista):
        return listaVacia
    else:
        return lista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))

# Tests
valores = lista(1, lista(2, lista(3, lista(4, listaVacia))))
assert mapa(lambda x: 2*x, valores) == lista(2, lista(4, lista(6, lista(8, listaVacia))))
```

CC1002 Introducción a la
Programación

Función filtro

- Recibe una lista y una función que devuelve un *boolean*
- Devuelve una lista con los elementos de la lista original para los que la función devuelve `True`
 - Filtra los elementos para los que la función devuelve `False`

CC1002 Introducción a la
Programación

Función filtro

- Implementación con contrato general:

```
# filtro: (X -> bool) lista(X) -> lista(X)
# devuelve lista con todos los valores de unaLista
# en donde la funcion operador devuelve True
def filtro(operador, unaLista):
    if vacia(unaLista):
        return listaVacia
    else:
        if operador(cabeza(unaLista)):
            return lista(cabeza(unaLista), filtro(operador, cola(unaLista)))
        else:
            return filtro(operador, cola(unaLista))

# Tests
valores = lista(6, lista(4, lista(8, listaVacia)))
assert filtro(lambda x: x < 5, valores) == lista(4, listaVacia)
valores = lista('a', lista('b', lista('c', lista('d', listaVacia))))
assert filtro(lambda x: x >= 'b' and x < 'd', valores) == lista('b', lista('c', listaVacia))
```

CC1002 Introducción a la
Programación

Función `fold`

- Otros problemas con listas:
 - Sumar todos los valores de una lista
 - Concatenar todas las palabras de una lista
- Estos problemas implican procesar la lista para obtener un único valor
 - Función **fold** (“reducir”): recibe una lista y una función de dos argumentos, procesa los elementos de la lista y devuelve un único valor

CC1002 Introducción a la
Programación

Función `fold`

- Implementación con contrato general:

```
# fold: (X X -> X) X lista(X) -> X
# procesa la lista con la función f y devuelve un único valor
# el valor init se usa como valor inicial para procesar el primer valor de la lista
# y como acumulador para los resultados parciales.
# Si la lista está vacía, se devuelve init
def fold(f, init, unaLista):
    if vacia(unaLista):
        return init
    else:
        return fold(f, f(init, cabeza(unaLista)), cola(unaLista))

# Tests
valores = lista(1, lista(2, lista(3, lista(4, listaVacía))))
assert fold(lambda x, y: x + y, 0, valores) == 10
valores = lista('pedro', lista('juan', lista('diego', listaVacía)))
assert fold(lambda x, y: x + y, '', valores) == 'pedrojuandiego'
```

CC1002 Introducción a la
Programación

Funciones abstractas para listas

- Resumen funciones abstractas para listas:
 - `mapa`
 - `filtro`
 - `fold`
- Permiten resolver problemas complejos
 - Ej.: lista de números, calcular la suma de los cuadrados de los valores positivos de la lista

```
fold(lambda x, y: x + y, 0, mapa(lambda x: x**2, filtro(lambda x: x > 0, valores)))
```

CC1002 Introducción a la
Programación

Para la próxima clase

- Repasaremos los contenidos de la Unidad II del curso

CC1002 Introducción a la
Programación